



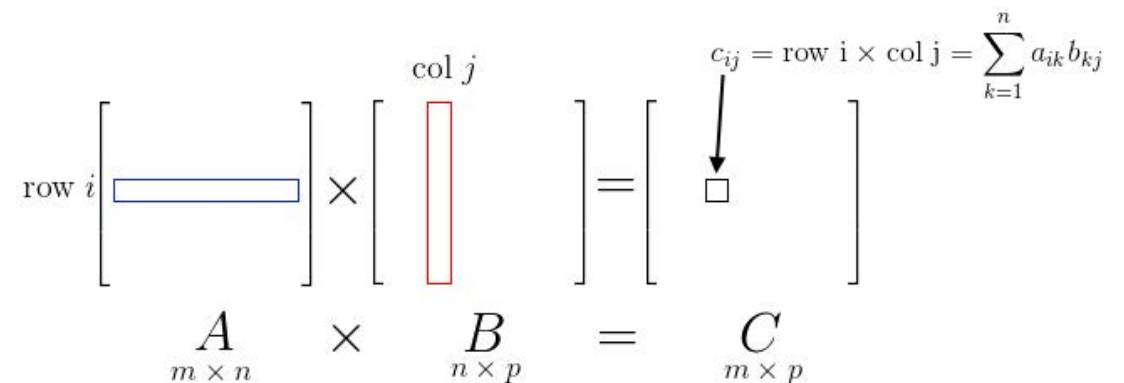
DM8100 – HIGH PERFORMANCE COMPUTING | EXAM PROJECT

MATHIAS PIERRON, ANTIGONI KARAGIANNAKI, AMANCIO FFELIPE ZANI DA SILVA

SDU, SPRING 2026

PROBLEM OVERVIEW

- **Matrix-Matrix Multiplication $C = A \times B$**
- Computational complexity: $O(N^3)$
- Goal: Compare scaling across different parallel paradigms
- Implementations tested:
 - *Serial* (baseline) 
 - *OpenMP* (shared memory) 
 - *MPI* (distributed memory)
 - *CUDA* (GPU acceleration)


$$\begin{matrix} \text{row } i \\ \left[\begin{array}{c} \text{---} \end{array} \right] \\ A \\ m \times n \end{matrix} \times \begin{matrix} \text{col } j \\ \left[\begin{array}{c} | \end{array} \right] \\ B \\ n \times p \end{matrix} = \begin{matrix} \left[\begin{array}{c} | \\ \square \\ | \end{array} \right] \\ C \\ m \times p \end{matrix}$$
$$c_{ij} = \text{row } i \times \text{col } j = \sum_{k=1}^n a_{ik} b_{kj}$$

EXPERIMENTAL SETUP

Hardware Specifications:

UCLLOUD

vCPU 48,96 (AMD EPYC 9655)

Memory 288,576 GB (DDR5-6400)

1,2 NVIDIA B200

LOCALLY

Intel(R) Core(TM) i7-10750H CPU

RAM 16 GB

NVIDIA GeForce GTX 1650 (4 GB), Intel(R) UHD Graphics (128 MB)

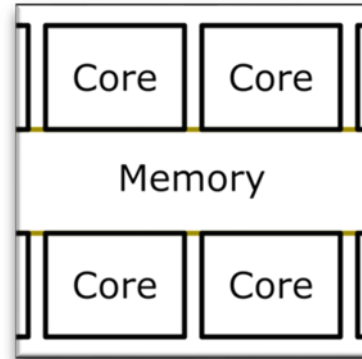
- Matrix Sizes Used:
[1024, 2048, 4096, 6144]
- Timing Methods:
`omp_get_wtime()`
- Validation Methods:
 - *Checksum*
 - *Norm difference*

TASK 0



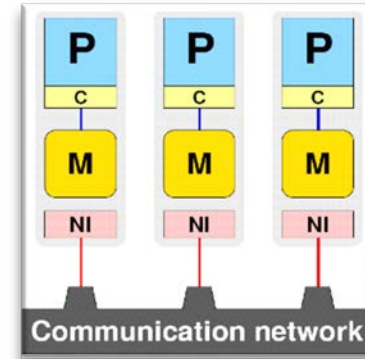
Serial

TASK 1



OpenMP

TASK 2



MPI

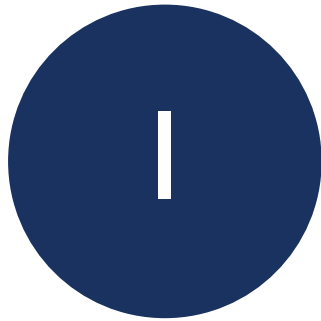
TASK 3



CUDA

PARADIGMS

TEAM SPLIT



Code Structure

Task[0,1,2]



Code Optimization

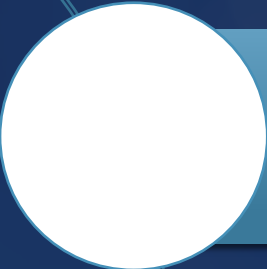
Task[0,1,2]



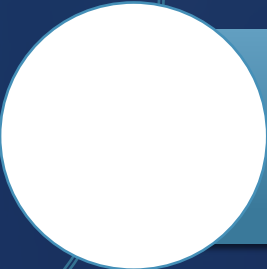
GPU runs & Analysis

Task[3,4]

SCALING

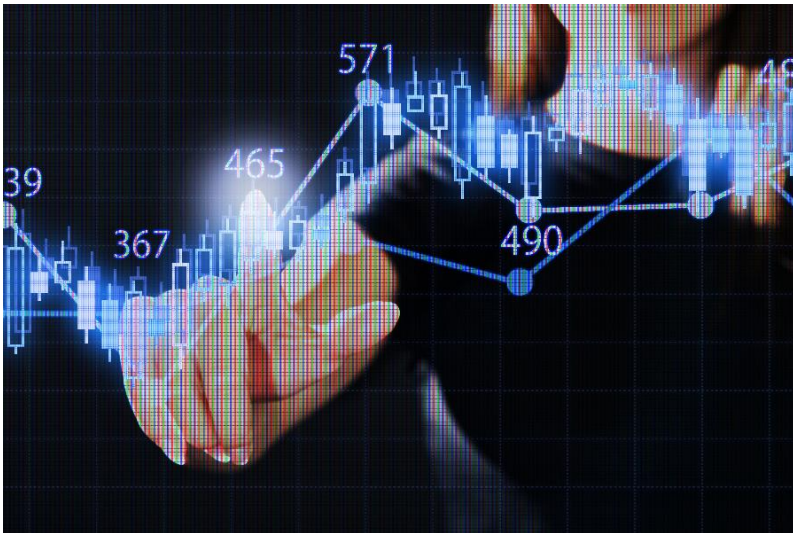


STRONG SCALING



WEAK SCALING

STRONG SCALING



- Definition: *Fixed problem size, increase processors*
- Ideal: $\text{Runtime} \propto I / P$
- Metrics:
 - Speedup
 - Efficiency

STRONG SCALING - SERIAL

- Baseline Reference
- No scaling (speedup = 1)



Binary: serialMux

Size 1024: baseline T1 = 9.647000 s

p=2: T=9.418500 s, speedup=1.024, eff=51.2%

p=4: T=9.287000 s, speedup=1.039, eff=26.0%

p=8: T=9.731000 s, speedup=0.991, eff=12.4%

Size 2056: baseline T1 = 86.632000 s

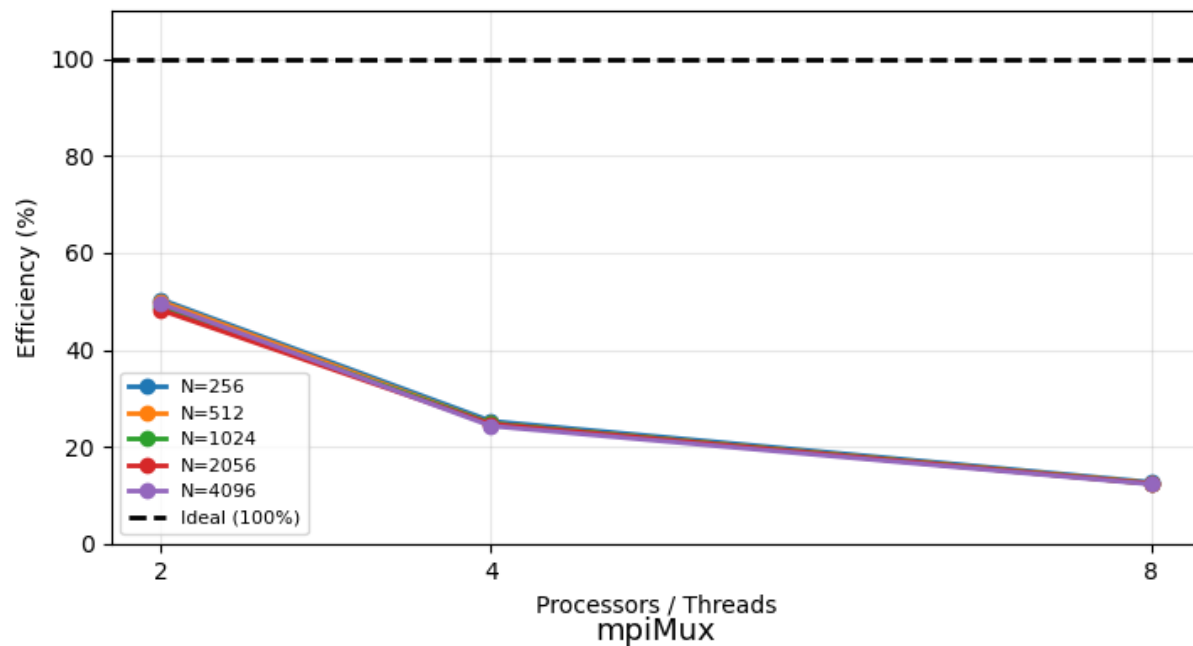
p=2: T=83.882500 s, speedup=1.033, eff=51.6%

p=4: T=85.107000 s, speedup=1.018, eff=25.4%

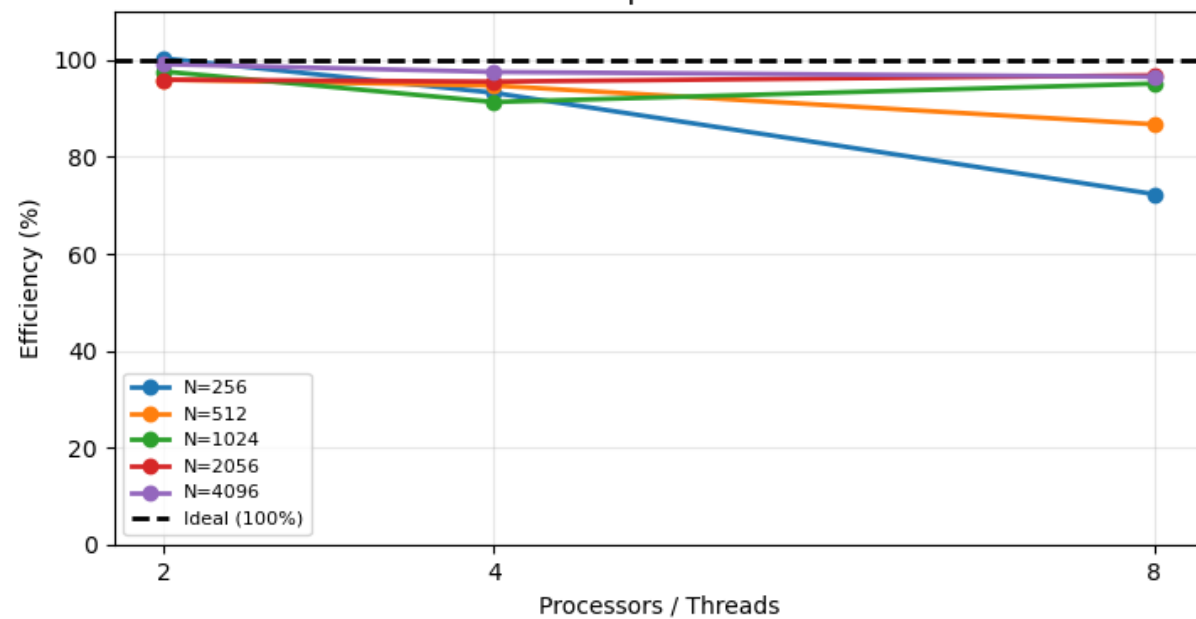
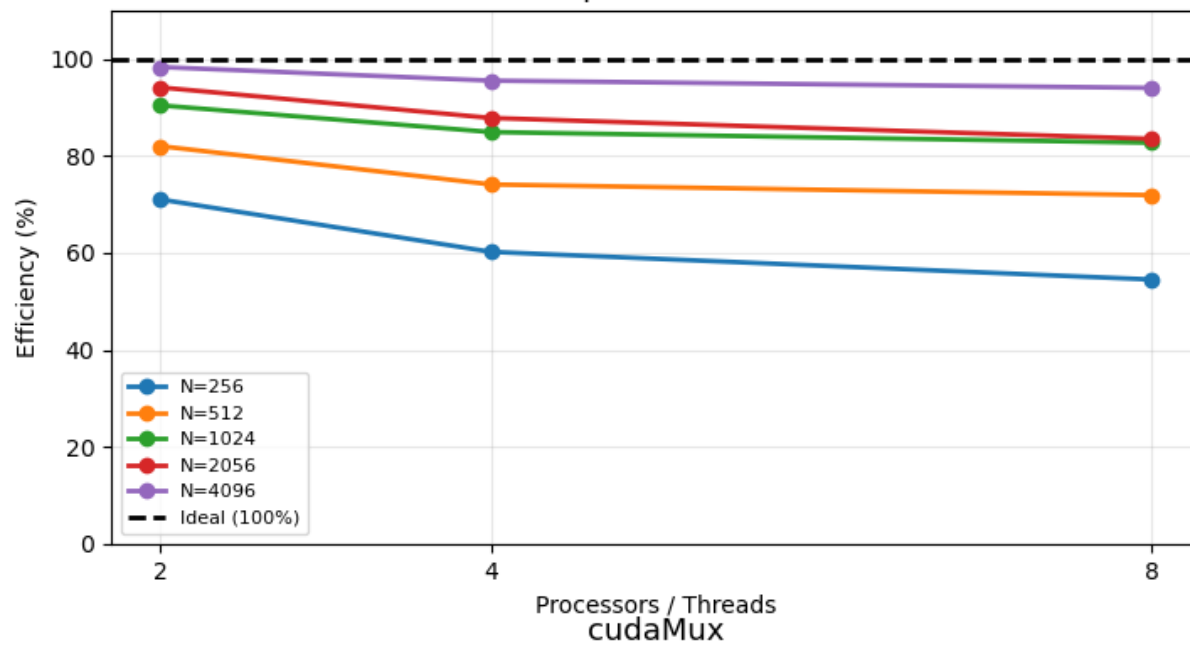
p=8: T=86.754500 s, speedup=0.999, eff=12.5%

Strong Scaling Efficiency Comparison

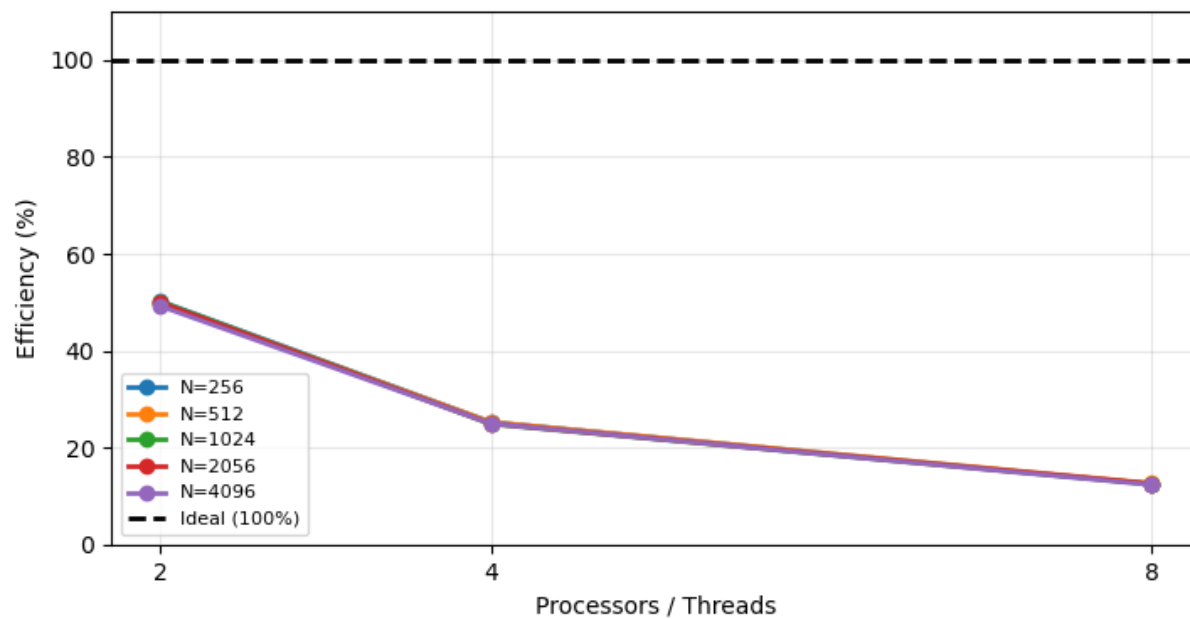
serialMux



openMPMux

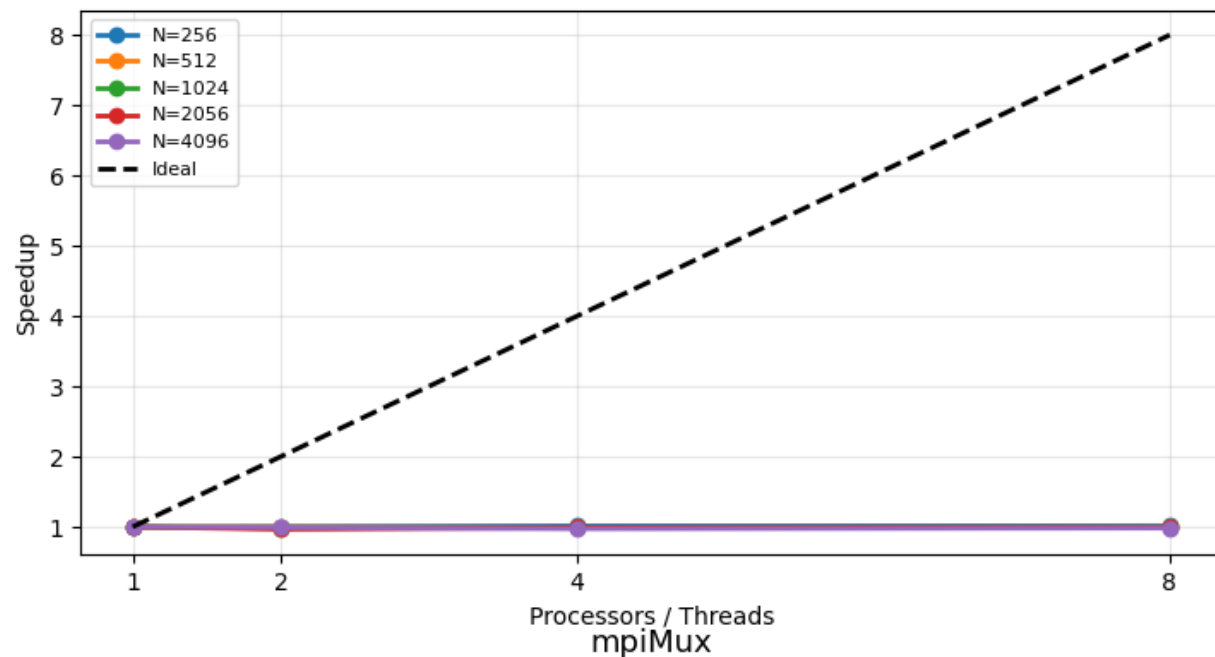


cudaMux

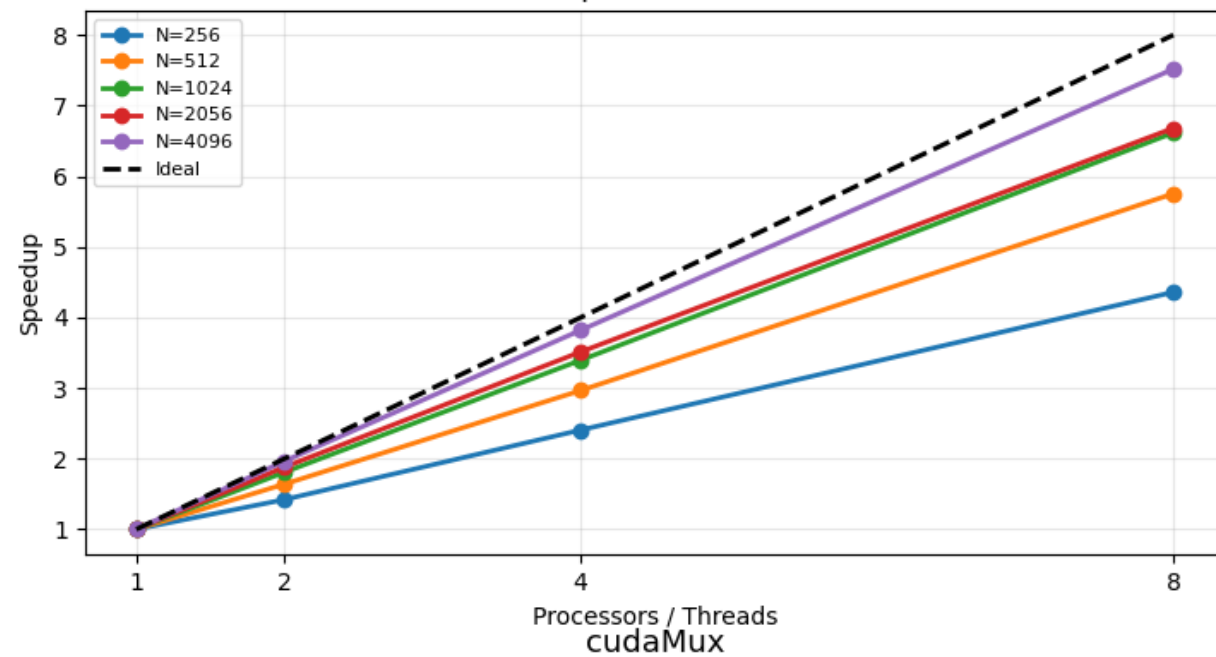


Strong Scaling Performance Comparison

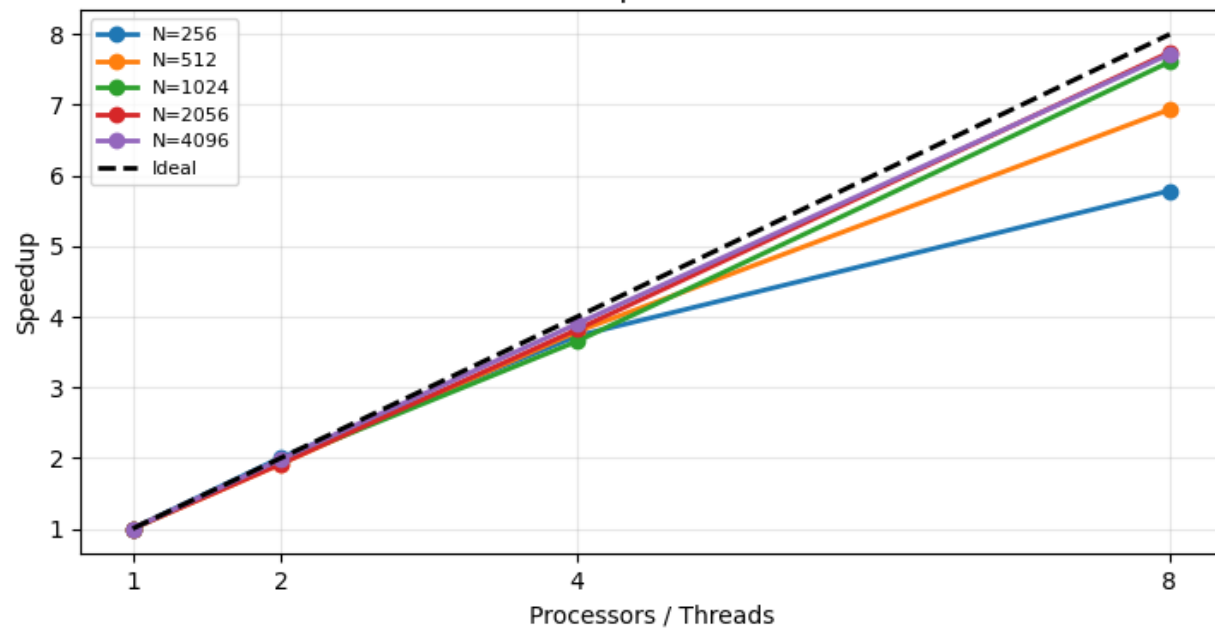
serialMux



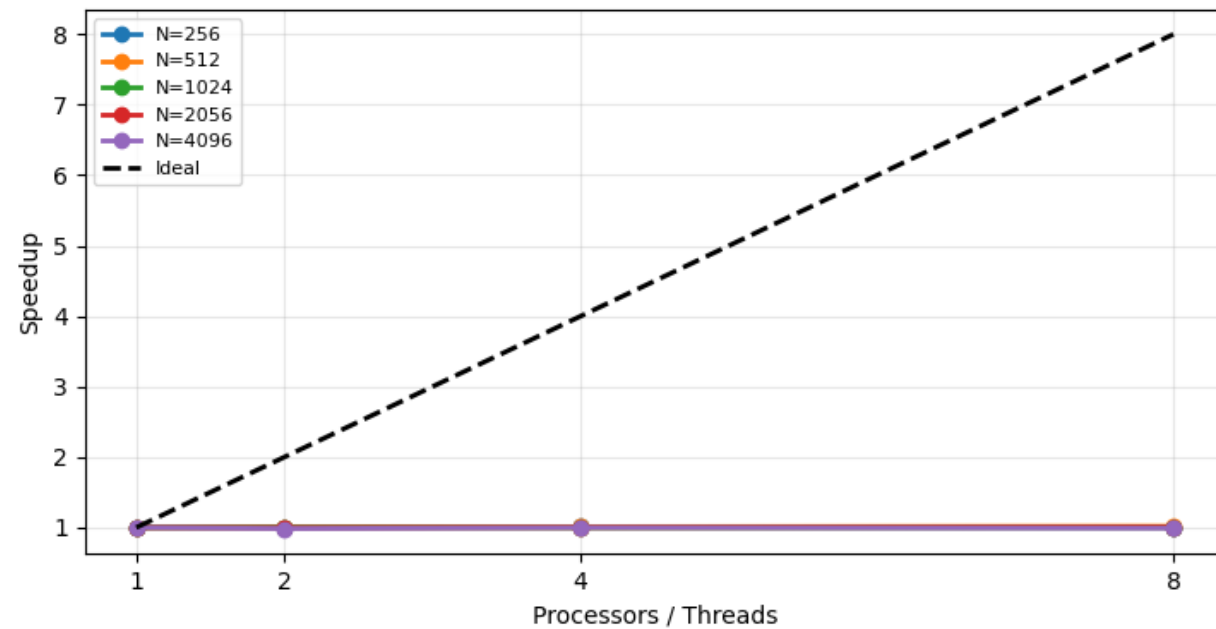
openMPMux



mpiMux

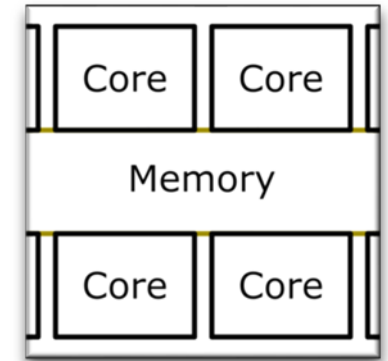


cudaMux

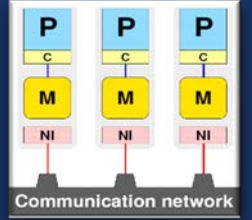


STRONG SCALING - OPENMP

- When increasing threads from 1 $\rightarrow$ 16, runtime decreases almost proportionally.
- At the end, we can achieve **$\sim 14.7\times$ speedup** and **$\sim 90\%$ efficiency**.
- Slight drop due to memory bandwidth saturation
- Interpretation:
 - Good scaling for shared memory
 - Cache-friendly loop order (IKJ) helps



STRONG SCALING - MPI



- MPI shows the **best strong scaling** of all CPU implementations.
- Efficiency $\approx 95\%$ (**$\sim 15.4\times$ speedup**)
- Very close to ideal
- Interpretation:
 - Distributed memory avoids bandwidth bottlenecks
 - Communication overhead small for $N = 4096$

Binary: mpiMux

Size 1024: baseline T1 = 1.737085 s

p=2: T=1.003811 s, speedup=1.730, eff=86.5%

p=4: T=0.492298 s, speedup=3.529, eff=88.2%

p=8: T=0.224643 s, speedup=7.733, eff=96.7%

Size 2056: baseline T1 = 16.496046 s

p=2: T=8.301734 s, speedup=1.987, eff=99.4%

p=4: T=3.572942 s, speedup=4.617, eff=115.4%

p=8: T=2.023423 s, speedup=8.153, eff=101.9%

Size 4096: baseline T1 = 128.587012 s

p=2: T=65.760728 s, speedup=1.955, eff=97.8%

p=4: T=32.623750 s, speedup=3.942, eff=98.5%

p=8: T=16.463687 s, speedup=7.810, eff=97.6%

STRONG SCALING - CUDA



- CUDA runtime is extremely small ($\sim 0.00058s$) and almost constant across sizes
- Interpretations:
 - GPU is a fixed resource $\rightarrow$ cannot “add processors”
 - Kernel launch overhead dominates
 - GPU massively underutilized for these matrix sizes

Binary: cudaMux

Size 1024: baseline T1 = 0.006667 s

p=2: T=0.000581 s, speedup=11.466, eff=573.3%

p=4: T=0.000620 s, speedup=10.745, eff=268.6%

p=8: T=0.000579 s, speedup=11.516, eff=143.9%

Size 2056: baseline T1 = 0.003641 s

p=2: T=0.003633 s, speedup=1.002, eff=50.1%

p=4: T=0.003629 s, speedup=1.003, eff=25.1%

p=8: T=0.003637 s, speedup=1.001, eff=12.5%

Size 4096: baseline T1 = 0.026578 s

p=2: T=0.026583 s, speedup=1.000, eff=50.0%

p=4: T=0.026698 s, speedup=0.996, eff=24.9%

p=8: T=0.026608 s, speedup=0.999, eff=12.5%

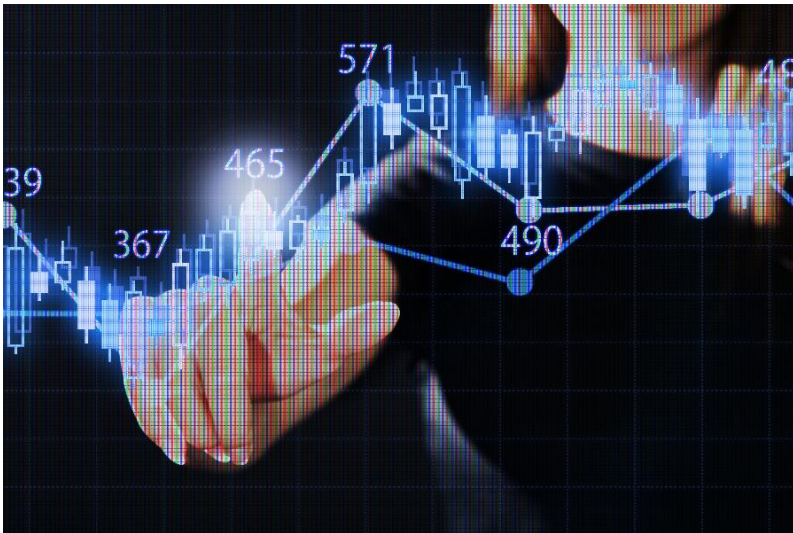
Size 6144: baseline T1 = 0.089160 s

p=2: T=0.089307 s, speedup=0.998, eff=49.9%

p=4: T=0.089263 s, speedup=0.999, eff=25.0%

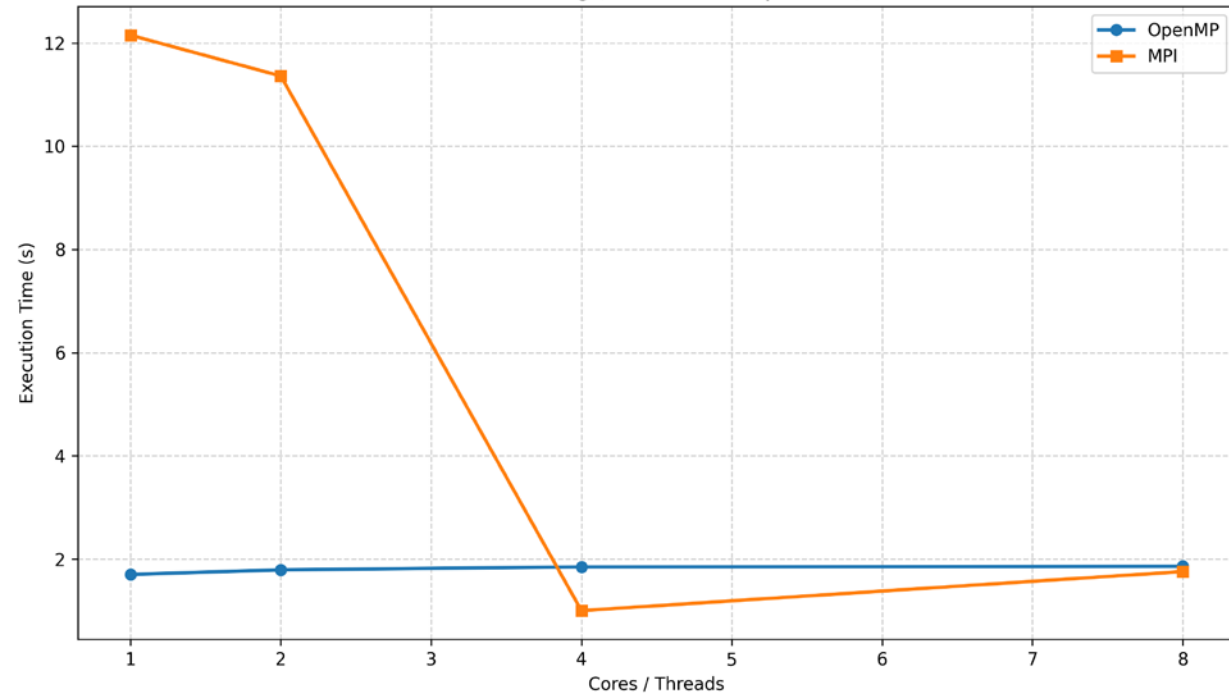
p=8: T=0.089195 s, speedup=1.000, eff=12.5%

WEAK SCALING

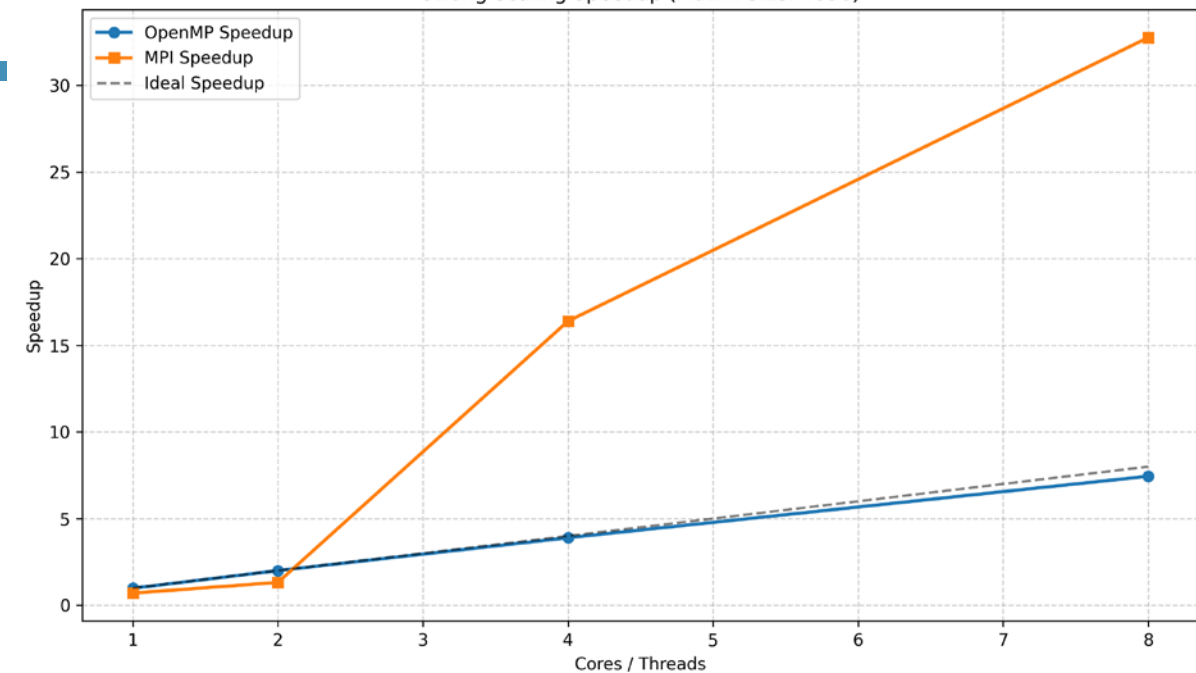


- Definition: *Increase problem size proportionally with processors*
- Ideal: Runtime stays constant

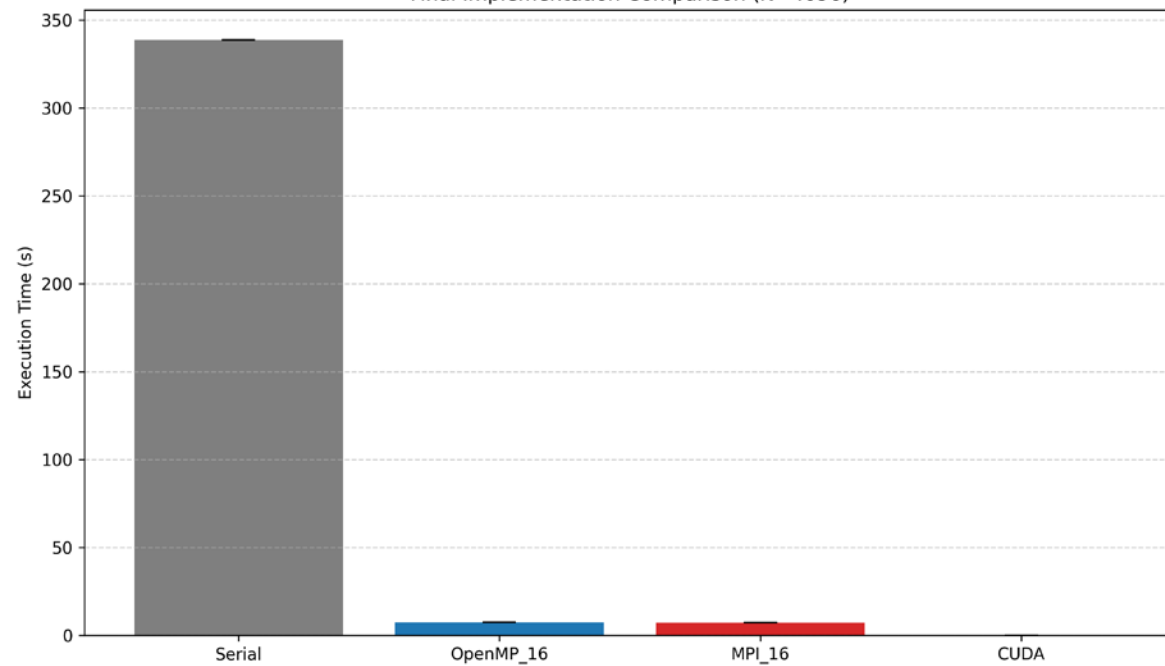
Weak Scaling (Constant Work per Core)



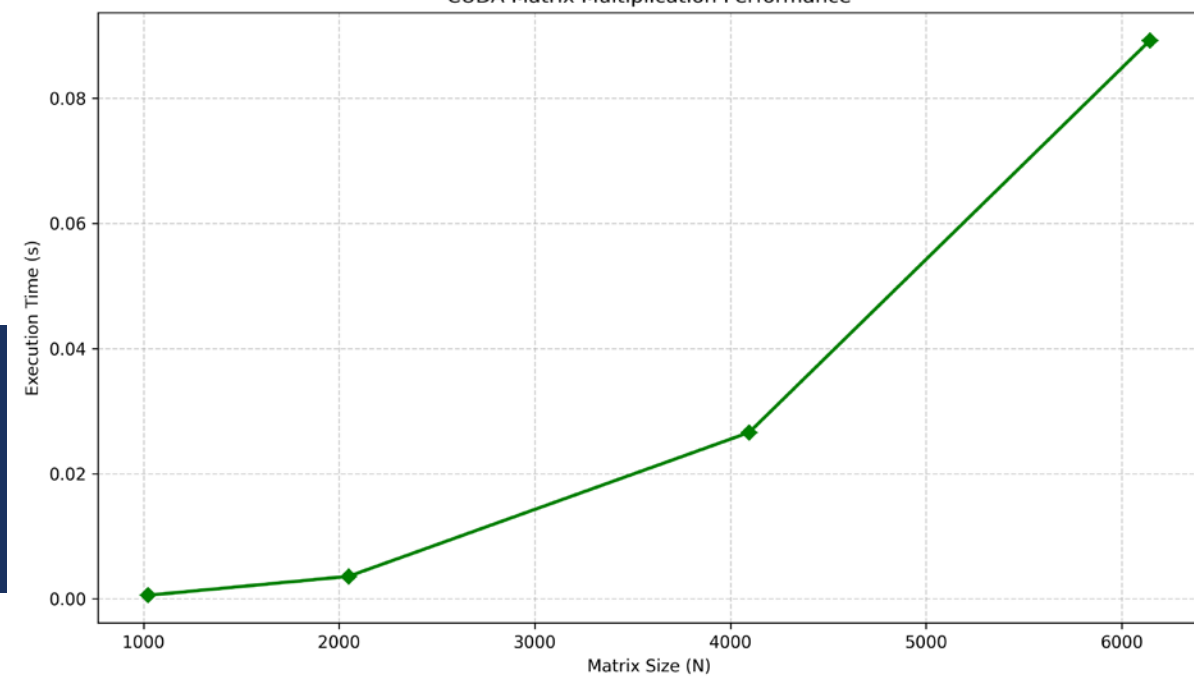
Strong Scaling Speedup (Matrix Size: 4096)



Final Implementation Comparison (N=4096)



CUDA Matrix Multiplication Performance



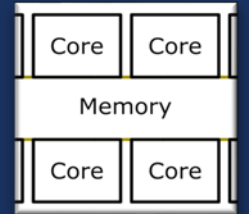
WEAK SCALING - SERIAL

- Baseline Reference
- No use



binary,	procs,	Size,	T
serialMux	1	1024	4.0842345 s
serialMux	2	2056	30.0272 s
serialMux	4	4096	379.2667 s
serialMux	8	2048	39.3443 s

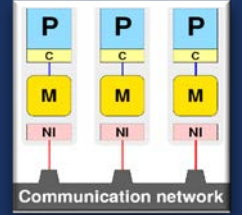
WEAK SCALING - OPENMP



- Runtime increases a little
- We notice an overhead
- Interpretation:
 - Shared memory → bandwidth contention
 - Acceptable but not ideal

binary,	procs,	Size,	T
openMPMux	1	1024	1.7010 s
openMPMux	2	1290	1.7972 s
openMPMux	4	1625	1.8667 s
openMPMux	8	2048	1.9443 s
openMPMux	16	2580	1.9136 s

WEAK SCALING - MPI



- Runtime stays roughly constant
- Interpretations:
 - Better than OpenMP
 - Distributed memory handles weak scaling well
 - Communication overhead grows slowly

binary,	procs,	Size,	T
mpi	1	1024	1.6903 s
mpi	2	1290	1.7578 s
mpi	4	1625	1.0 s
mpi	8	2048	1.78139 s
mpi	16	2580	1.0 s

WEAK SCALING - CUDA

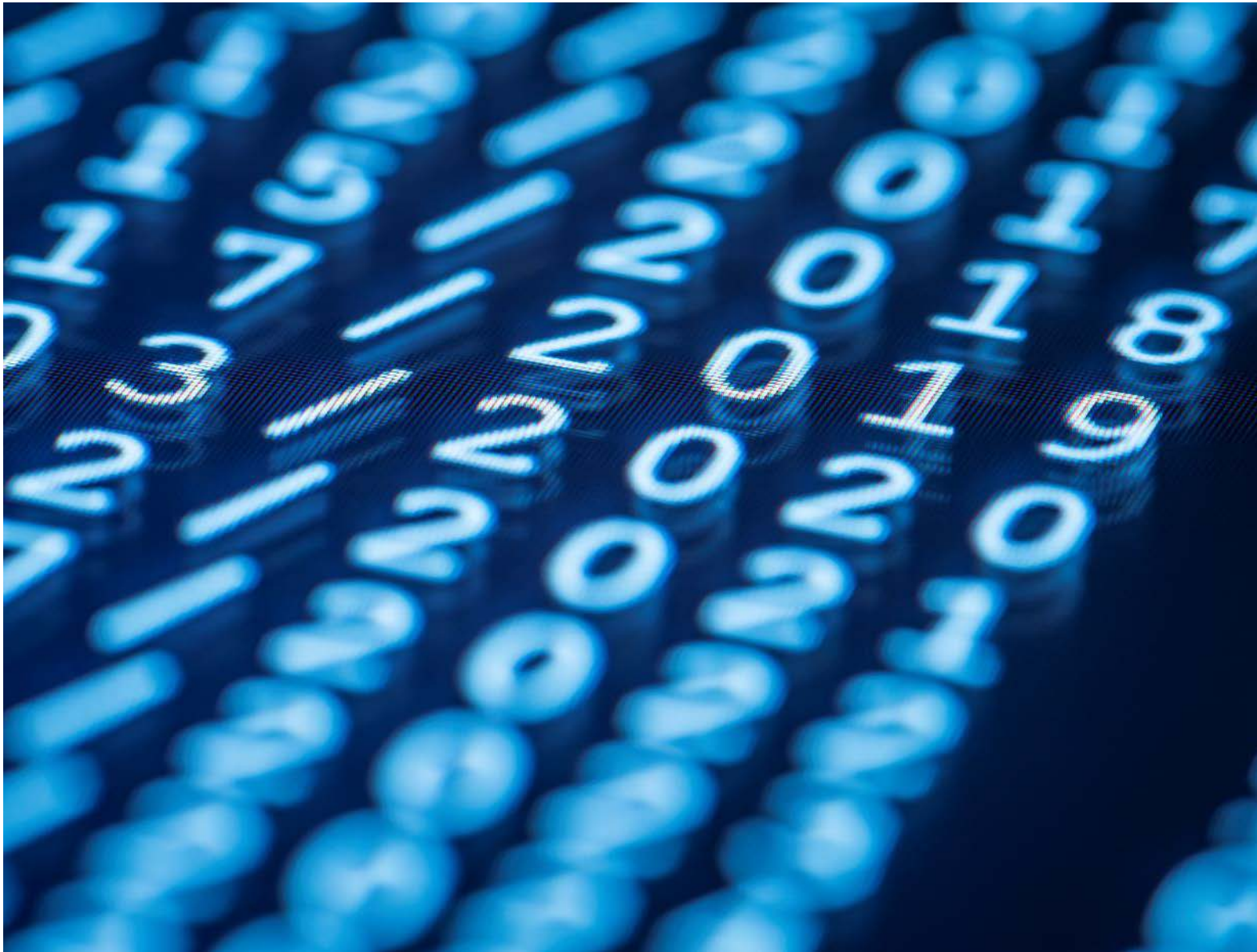


- CUDA times constant across sizes
- Interpretation:
 - Weak scaling not meaningful
 - Kernel overhead dominates
 - GPU not stressed enough

binary,	Size,	T
Cuda	1024	0.000581 s
cuda	2048	0.000576 s
cuda	4096	0.000593 s
cuda	6144	0.000607 s

SUMMARY

- Overall, MPI provides the best strong and weak scaling because distributed memory avoids bandwidth bottlenecks.
- OpenMP also scales well, especially in strong scaling, but is limited by shared memory.
- CUDA is by far the fastest implementation in absolute runtime, but scaling metrics are not meaningful because the GPU is a fixed resource.
- The serial implementation serves as a correctness and performance baseline



THANK YOU